Anamnésia: An Agent-Independent Memory Gateway for Persistent Context Retrieval, Token Optimization, and Cost-Aware AI Systems

Gabriel (GabezFC)
Anamnésia project (independent)
Technical report v2.1.0 (independent; not an IEEE publication)
Prepared with the Hermes agent, 25 October 2026
Repository: `GabezFC/Anamnesia`

Abstract

Coding agents and chat assistants repeatedly pay to re-read the same notes, decisions and project facts, and lose that knowledge when a session ends. Anamnésia is an open, agent-independent *memory gateway*: a local service that sits between an agent and a Markdown/Obsidian vault, retrieves the few passages relevant to a query with SQLite FTS5/BM25 (optionally enriched by a Graphify knowledge graph and a paid relevance judge, JEV), trims and de-duplicates them under a token budget, and returns an auditable, injection-flagged context through the Model Context Protocol (MCP) or a REST API, while recording tokens and cost in a ledger. This paper (i) documents the implemented architecture from the source code, (ii) separates original contributions from functionality supplied by external components, and (iii) reports a real, previously executed generative benchmark: 82 questions over a sanitized slice of the author's own vault, nine models (eight remote through OpenRouter, one local through Ollama), five context pipelines, and 3 797 logged generation calls. Compared with a whole-note BM25 baseline, the default pipeline (baseline retrieval plus the optimizer) cut provider-reported input tokens from 11,103 to 1,776 per query and the effective cost of the eight remote models by 81.7%, while pooled accuracy as scored by an LLM judge was statistically indistinguishable (86.9% vs. 86.6%; paired $\Delta{=}+0.3$ pp, 95% CI $[-7.4, +8.0]$). Three negative or qualifying findings are equally important: the Graphify-hybrid pipeline lost 15.0 pp of accuracy; the paid JEV judge added cost without any accuracy gain; and a stratified re-analysis shows that the headline parity hides a trade-off—on questions where the baseline retrieved a context, Anamnésia was 6.9 pp *less* accurate, whereas it was far better on questions where the baseline's abstention gate returned nothing. Persistence of memory across real agent sessions and recall after long conversations were *not* measured; we state them as unproven. Aggregate figure data, plotting scripts and bilingual figures are released with the paper; raw experiment logs remain private.

Index Terms

memory gateway, retrieval-augmented generation, Model Context Protocol, BM25, knowledge graph, token optimization, LLM cost accounting, agent memory

I. Introduction

Large-language-model (LLM) agents are stateless between sessions and pay per token within them. Two practical problems follow. First, *continuity*: decisions, facts and conventions established in earlier sessions must be re-supplied, usually by pasting notes or whole files into the prompt. Second, *cost*: input tokens dominate the bill of retrieval-heavy workloads, and long contexts can also degrade answer quality [1]. A well-known response is retrieval-augmented generation (RAG) [2], [3]; agent-specific memory systems extend it with writing, consolidation and paging [4]–[7].

Anamnésia (formerly *Memory Gateway*) targets a narrow, practical point in this design space: a *local*, *agent-independent*, *auditable* gateway for a human-curated Markdown vault. It does not train models and it does not require embeddings. Retrieval is lexical (SQLite FTS5 with BM25 ranking [8], [9]), optionally combined with a code/document knowledge graph produced by the external Graphify tool [10] and with a paid relevance/injection judge, *JEV*, offered by TypeSafe AI [11]. Results are delivered over MCP [12] or REST (FastAPI [13]) so that any agent that speaks MCP—Hermes, Claude Code, Codex, among others—can use the same memory.

Contributions. (1) A source-level description of the implemented system (Sections III–V), with eight diagrams that mirror the code paths. (2) A cost model that charges every stage, including the JEV judge, to the pipeline

that uses it (Section IV-E). (3) A documented generative benchmark with public aggregate figure data and plotting scripts (raw benchmark logs remain local and unpublished), including negative results (Sections VI–VII). (4) An explicit separation between demonstrated benefits and capabilities that remain unproven (Sections IX, X).

What this paper does not claim. It does not claim superiority over dense retrievers, graph-RAG systems or commercial memory products (none were evaluated), and it does not claim that cross-session agent memory improves task success (not measured).

II. Background and Related Work

RAG and lexical retrieval. RAG conditions generation on retrieved passages [2]; surveys catalogue chunking, retrieval and reranking choices [3]. Dense retrievers [14] dominate many open-domain benchmarks, but BM25 [9] remains a strong, training-free baseline, especially for small private corpora with exact identifiers (test names, flags, version numbers), which are common in engineering notes. Rank-fusion methods such as RRF [15] combine lexical and other signals; Anamnésia implements an RRF mode but disables it by default.

Graph-based retrieval. GraphRAG builds an entity graph and community summaries for query-focused summarization [16]; HippoRAG applies personalized PageRank over a knowledge graph for long-term memory [17], [18]. The Graphify tool used here is different in kind: it maps a folder to a graph with tree-sitter ASTs for code and assistant-driven extraction for documents, and its edges are tagged as extracted or inferred [10]. As measured by the Anamnésia authors on 27 Sep 2026, the graph produced from the vault contains only page and heading labels, never body text (Section IV-C).

Agent memory. MemGPT manages a hierarchy of memory tiers [4]; Generative Agents store and reflect on experience streams [5]; MemoryBank adds forgetting curves [6]; Mem0 extracts and consolidates facts [7]. LoCoMo and LongMemEval measure long-horizon conversational memory [19], [20]. Anamnésia differs in being agent-independent and file-based (notes remain ordinary Markdown), and it has *not* been evaluated on these benchmarks.

Context and cost optimization. Prompt compressors such as LLMLingua [21], [22] shorten text with a small LM; FrugalGPT and RouteLLM reduce spend by cascading or routing among models [23], [24]. Anamnésia reduces cost by *sending less context*; its routing and compression modules exist but are optional and were not part of the benchmark. Near-duplicate removal uses SimHash and MinHash ideas [25]–[27].

LLM-as-a-judge. Using an LLM to grade free-form answers is common and carries known biases [28]; we use it, document a scoring defect it exhibited, and give a sensitivity analysis (Section VII-C).

III. System Architecture

Fig. 1 shows the system as implemented. Agents reach the gateway by MCP over stdio (default toolset: the single tool `memory_search`; the full set has seven tools, with `memory_save` and `memory_feedback` opt-in), by REST (about 130 operations, writes guarded by a loopback check, Origin/Referer matching and an `X-MG-Token`), or by a command-line tool (`anamnesia`: `doctor`, `start`, `mcp`). An optional MCP-to-REST bridge forwards only `memory_search`. All paths enter the same in-process object, `MemoryGateway`, whose search implementation executes the stages detailed in Section IV.

State. The user's vault is mirrored byte-for-byte into a data directory and opened read-only through confined `open(rb)` calls; NTFS alternate data streams and reserved device names are rejected. Notes written by agents are stored separately under `saved_context/` (Section V). A SQLite file (`benchmark.db`) holds the *ledger* (`runs`, `candidates`, `budgets`, `routing_decisions`, `verification_results`, `jev_cache`, and others); a second file holds usage statistics. Token and cost fields are not columns but keys of a JSON blob (`metrics_json`: `jev_input_tokens`, `jev_cost`, `context_tokens`, `total_tokens_spent`, ...).

Components and provenance. Table I separates what Anamnésia implements from what it consumes.

The Python dependency list of the repository is short (FastAPI, uvicorn, `mcp`, `httpx`, Pydantic [31], `typesafe-sdk`, PyYAML, `anthropic`); notably there is *no* separate BM25 package and *no* embedding model, and `tiktoken` [32] is not a runtime dependency (it was used only by the benchmark harness for an independent token count).

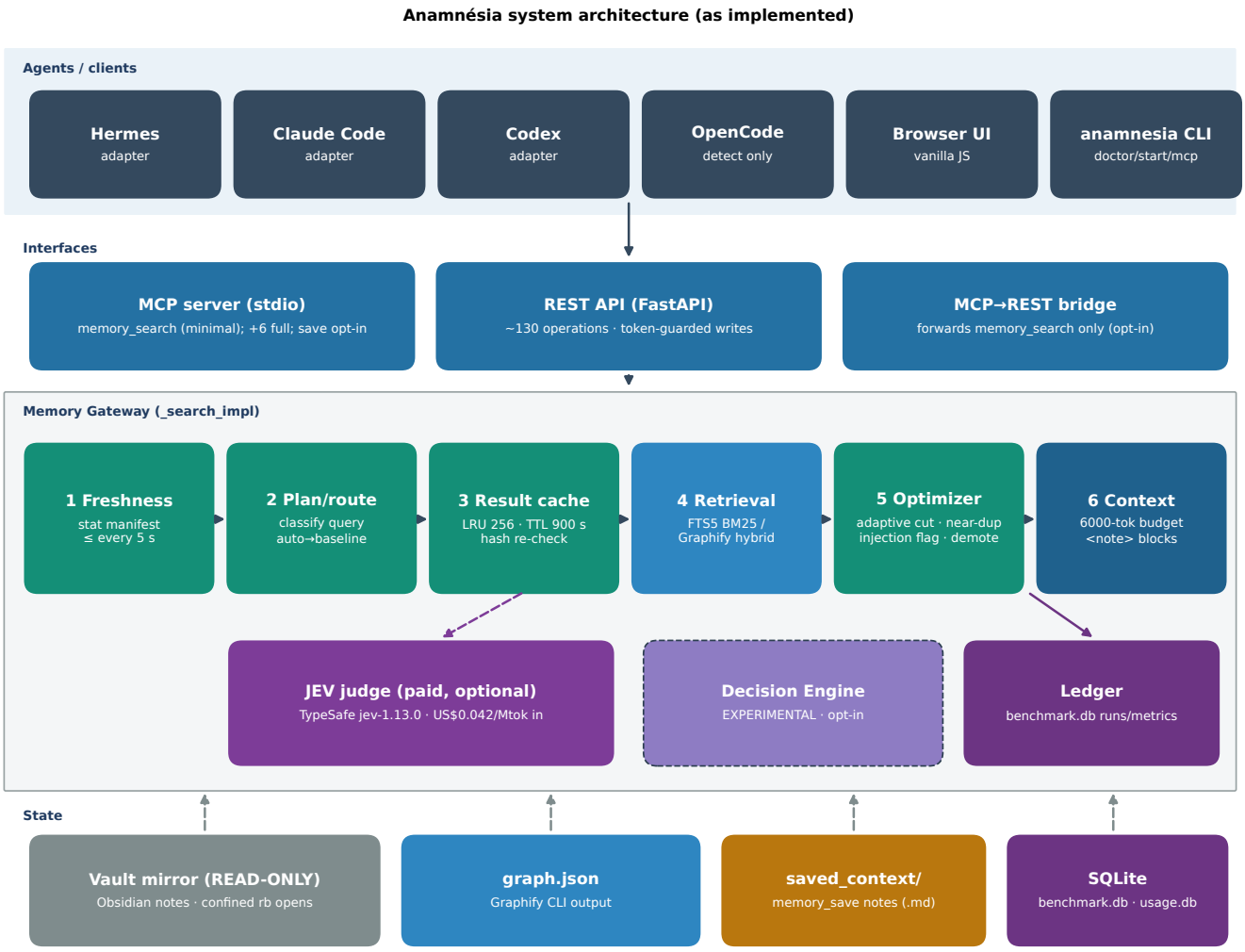


Fig. 1. System architecture as implemented. Dashed boxes are optional or experimental. The vault mirror is read-only; agent-written notes live in a separate `saved_context/` folder.

IV. Memory Retrieval and Optimization

A. *Query path*

Fig. 2 summarizes the stages of a search; their order is fixed in the gateway: (1) a freshness check that compares a stat manifest of the vault at most every `MG_VAULT_REFRESH_S` = 5 s and rebuilds the index on change (clearing caches); (2) query classification into SIMPLE/MEDIUM/COMPLEX/AMBIGUOUS and routing (every class defaults to the `baseline` pipeline, so `pipeline=auto` resolves to baseline unless the opt-in *economy* policy selects a JEV pipeline); (3) optional budget guard; (4) result-cache lookup; (5) the retrieval pipeline; (6) scope re-check; (7) the post-filter; (8) context construction; (9) metric/ledger write and cache insertion.

B. *Lexical retrieval*

Notes are split into sections (heading plus body) and indexed in an FTS5 virtual table with the `unicode61 remove_diacritics 2` tokenizer. Ranking uses the built-in function `bm25(sec,0,w,1,0,0)` with a

TABLE I
ROLE AND PROVENANCE OF COMPONENTS. "ORIGINAL" MEANS IMPLEMENTED IN THE ANAMNÉSIA REPOSITORY.

Component	Role in Anamnésia	Provenance
SQLite FTS5	lexical index, `bm25()` ranking [8]	external
Graphify CLI	builds `graph.json` from the vault mirror [10]	external (Apache-2.0)
Hybrid retrieval	BM25 seeds + graph BFS expansion	original
Optimizer	cut, near-dup, injection flag, cache, budget	original
JEV (TypeSafe)	paid relevance/injection judge [11]	external service; client logic original
MCP server	tool surface for agents [12]	protocol external; server original
FastAPI/uvicorn	REST layer [13]	external
Ollama	local model runtime in benchmark [29]	external
Ledger/cost model	token and cost accounting	original
Laya	optional local relevance model [30]	external, opt-in
Decision Engine	provider-agnostic keep/drop framework	original, *experimental*

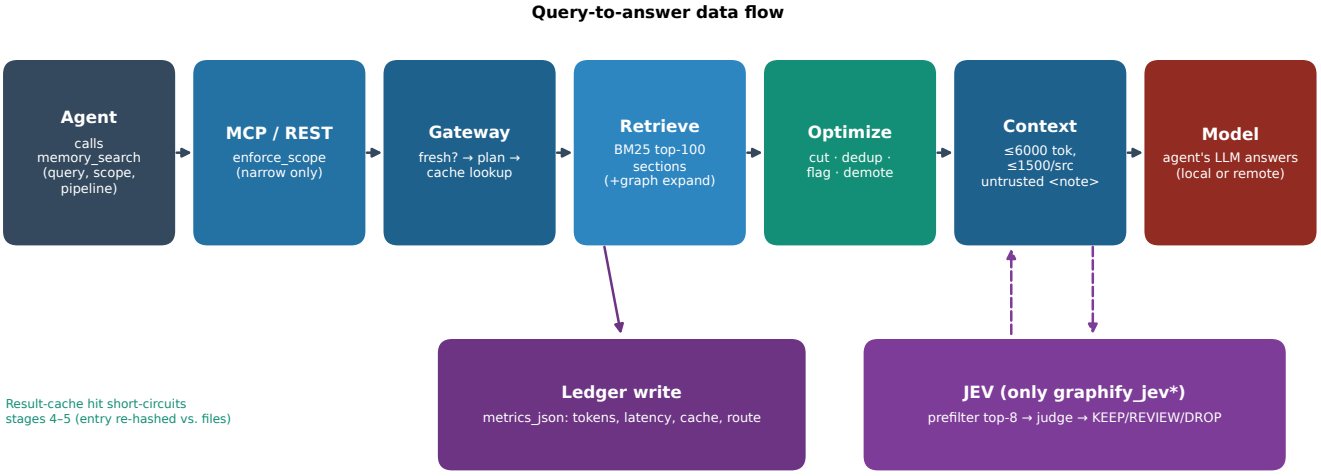


Fig. 2. Query-to-answer flow. JEV participates only in the `graphify_jev*` pipelines.

heading weight $w = 2.0$ and body weight 1. SQLite's `bm25()` implements the Okapi form [8], [9]

$$s(d,q) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \tag{1}$$

where SQLite fixes k_1=1.2 and b=0.75; the Anamnésia code does not override them (this is our inference from the code, not an explicit statement in it). SQLite returns negated scores, which Anamnésia reverses. Up to `MAX_CANDIDATES` = 100 sections are kept per query.

C. Graphify hybrid retrieval

The external Graphify CLI (`graphify update`, AST-only for this use) turns the vault mirror into `graph.json`. A design note in the repository records that, on the author's vault, this graph held 1,109 nodes (74 pages and 1,035 headings) and *no body text*, so that seeding from graph labels alone recovered 8 of 10 development questions and failed when the answer lived only in a note body. Anamnésia therefore implements its own *hybrid* search (Fig. 3): the first ten distinct BM25 files are text seeds; breadth-first expansion over the graph adds up to 15

related files. Seed candidates receive the rank-ladder score $1.0 - 0.45\,r/n_{\text{seeds}}$ (range $1.0 \to \approx 0.55$) and graph-only candidates $0.5 - 0.2\,i/n_{\text{rel}}$ ($0.5 \to 0.3$). Personalized PageRank expansion (α=0.85, 30 iterations, top-15) exists but is off by default. These scores are *ranks, not relevance probabilities*, a fact with consequences in Section VII.

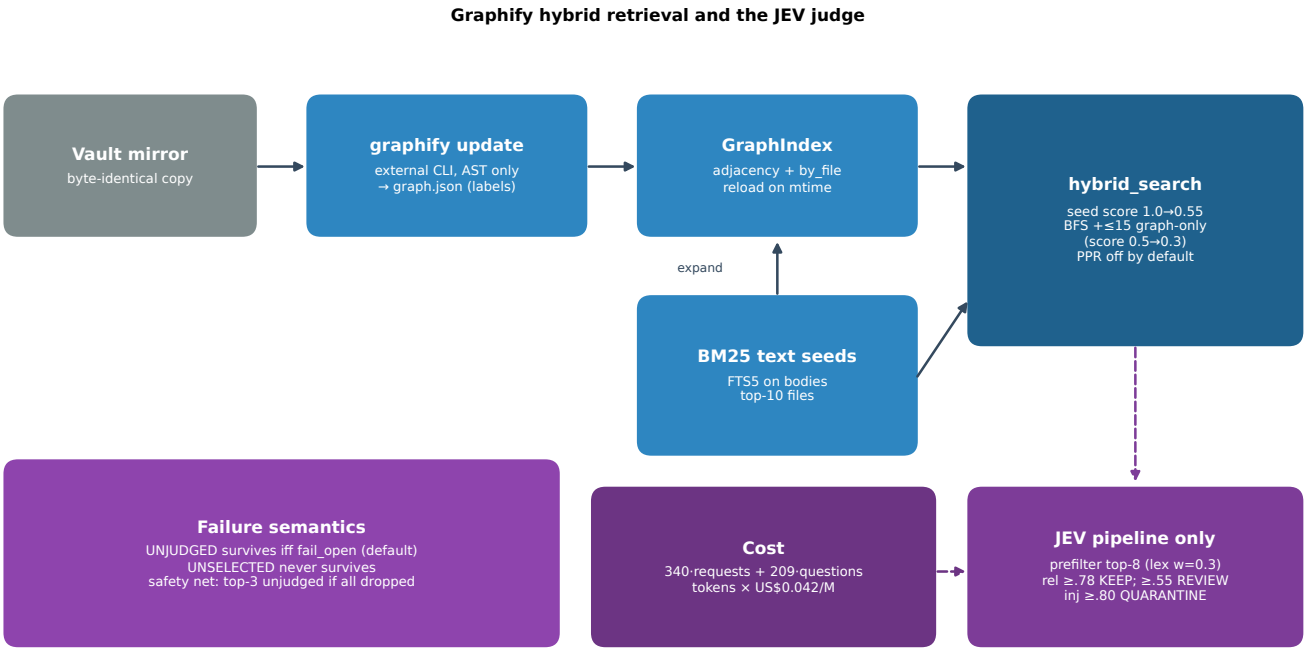


Fig. 3. Graphify hybrid retrieval and the JEV judge. The graph supplies structure; BM25 supplies content-addressable entry points.

D. Memory Optimization Layer

After retrieval, each candidate snippet is whitespace-normalized, truncated to `SNIPPET_MAX_TOKENS` = 300 and hashed. The post-filter (Fig. 4) applies, in order:

1) **Adaptive cut** (free pipelines only; SIMPLE/MEDIUM queries): keep candidate i if $i < 3$ or $\text{score}_i \geq 0.5 \cdot \max_j \text{score}_j$. Because baseline BM25 scores are unbounded while Graphify scores are a rank ladder, the same ratio acts differently in the two pipelines.
2) **Near-duplicate collapse**: 64-bit SimHash with 8 LSH bands (Hamming ≤ 12) and a 4-shingle Jaccard pre-gate select candidate pairs, confirmed by a sequence-similarity ratio ≥ 0.88 [25]–[27]. One section per note is kept by default (`DEDUP_MAX_PER_NOTE` = 1).
3) **Injection flagging**: a regular-expression screen ("inj-screen-v2", English and Portuguese, Unicode-normalized) marks suspicious notes with a warning attribute. It *flags but never drops*, and flagged results are not cached.
4) **Temporal and conflict demotion**: historical or conflicting notes have their score multiplied by 0.5.

The *context builder* then ranks candidates, interleaves them round-robin by file, caps each source at 1,500 tokens and fills a total budget of 6,000 tokens greedily, dropping a block if fewer than 120 tokens would remain. Each block is wrapped as a `<note source=...>` element and the whole context is declared untrusted.

Caching. The result cache is an in-process LRU (256 entries, 900 s TTL) keyed by the canonical query, pipeline, scope, budget, configuration digest and index epoch; on every hit the delivered files are re-hashed to detect staleness, and anti-poisoning rules refuse to cache flagged results. We found *no* embedding-based semantic cache for search results; the only query-normalized cache is the judge-verdict cache (query terms plus note content hash).

Token estimation. Without a tokenizer dependency, Anamnésia estimates $\hat{n} = \lceil \max(c/4,\ 1.3w) \cdot 1.13 \textit{ceil} \rceil$ from character and word counts, where 1.13 is a Portuguese calibration factor that the code itself describes as not freshly measured. All token counts in our results come from the *provider* (`usage.prompt_tokens`) or from `tiktoken` (context size), not from this estimator.

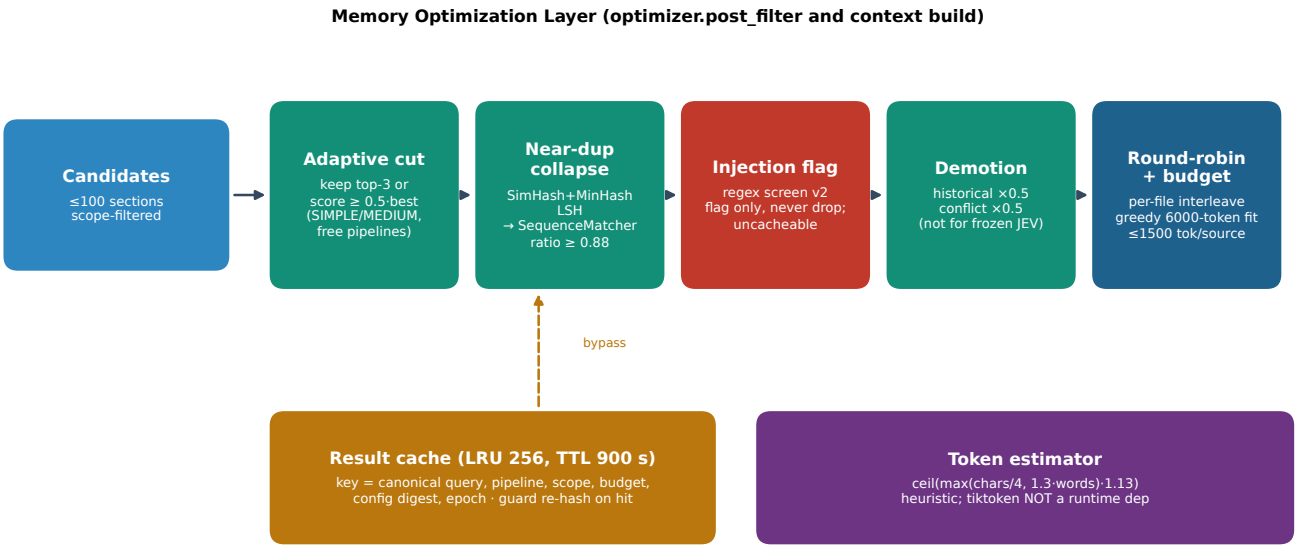


Fig. 4. Memory Optimization Layer: post-filter operations, result cache and token estimator.

E. JEV judge and cost model

In the `graphify_jev*` pipelines, a lexical prefilter keeps the top 8 candidates, which JEV (model `jev-1.13.0`) scores for relevance and injection. A candidate is `QUARANTINE`d if injection ≥ 0.80, `KEEP` if relevance ≥ 0.78, `REVIEW` if ≥ 0.55, else dropped; judge failures follow `fail_open` by default (unjudged candidates survive), and if everything is dropped the top 3 unjudged candidates are restored as a safety net. The repository's least-squares fit over ten real requests gives

$$\begin{aligned} \hat{n}_{\mathrm{JEV}} &\approx 340\,R + 209\,Q \quad \text{(input tokens)}, \\ C_{\mathrm{JEV}} &= \frac{n_{\mathrm{JEV}}}{10^6} \cdot 0.042 \text{ US\$}, \end{aligned} \tag{2}$$

for R HTTP requests and Q judged questions; output tokens are free at the quoted tariff. The effective cost of a pipeline that serves a model m is therefore

$$\begin{aligned} C_m^{\text{with}} &= \sum_q \Big[C_{\text{gen}}\big(m, \text{ctx}_q\big) + C_{\text{JEV}}(q) \Big], \\ C_m^{\text{without}} &= \sum_q C_{\text{gen}}\big(m, \text{notes}_q\big), \end{aligned} \tag{3}$$

with local retrieval and optimization costing zero dollars (CPU only; energy not measured). A saving is $1 - C^{\text{with}}/C^{\text{without}}$. Reduction of context alone is *not* a saving unless Eq. (3) confirms it; Fig. 5 shows what is charged. The experimental Decision Engine wraps JEV and other providers behind one interface with caching and per-operation fallback; it was not used in any experiment reported here, and its own telemetry labels JEV dollars as unknown.

V. Persistent Memory and Agent Integration

Storage model. With `MG_MCP_SAVE=1`, `memory_save` writes a Markdown note with YAML frontmatter (project, confidence, trust and feedback counters) into `saved_context/`, never into the user's vault. Writes are atomic (temporary file, `fsync`, `os.replace`) under a cross-process file lock (`msvcrt.locking` on Windows, `fcntl.flock` elsewhere). Saves pass a project gate, size and rate limits, and a secret scanner that rejects secret-like text. The folder is mounted as a read-only extra root of the index, so a refresh (Fig. 6) makes a saved note

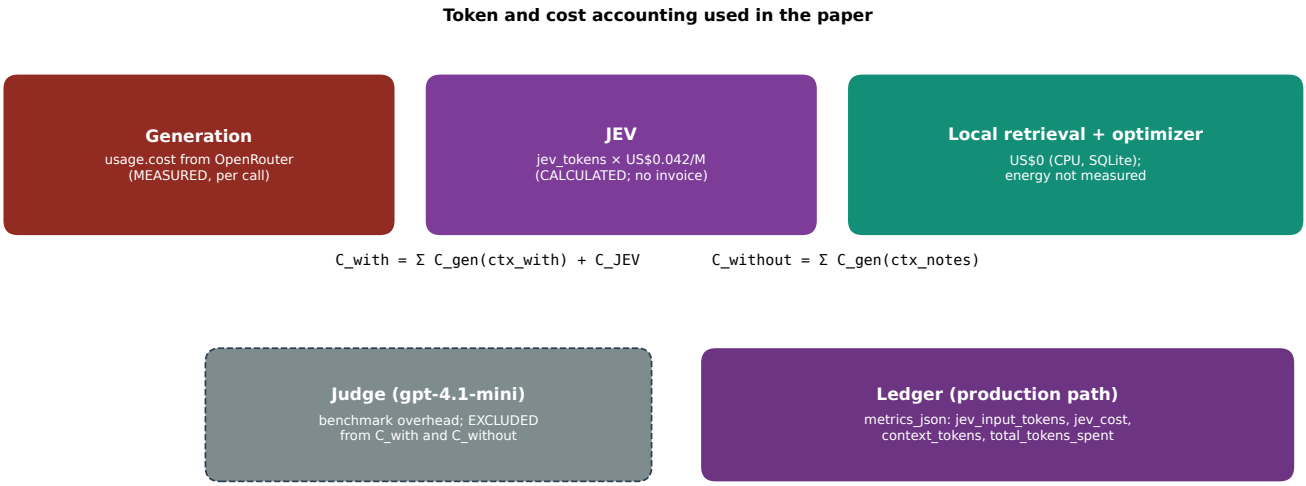


Fig. 5. Cost accounting used throughout. The evaluation judge is overhead of the experiment and is excluded from both arms.

retrievable by later searches, including in other processes and sessions. `memory_feedback` adjusts a note's confidence by $+0.05$ (useful), -0.2 (wrong) or -0.05 (stale); a composite rank ("v2") that uses it is opt-in.

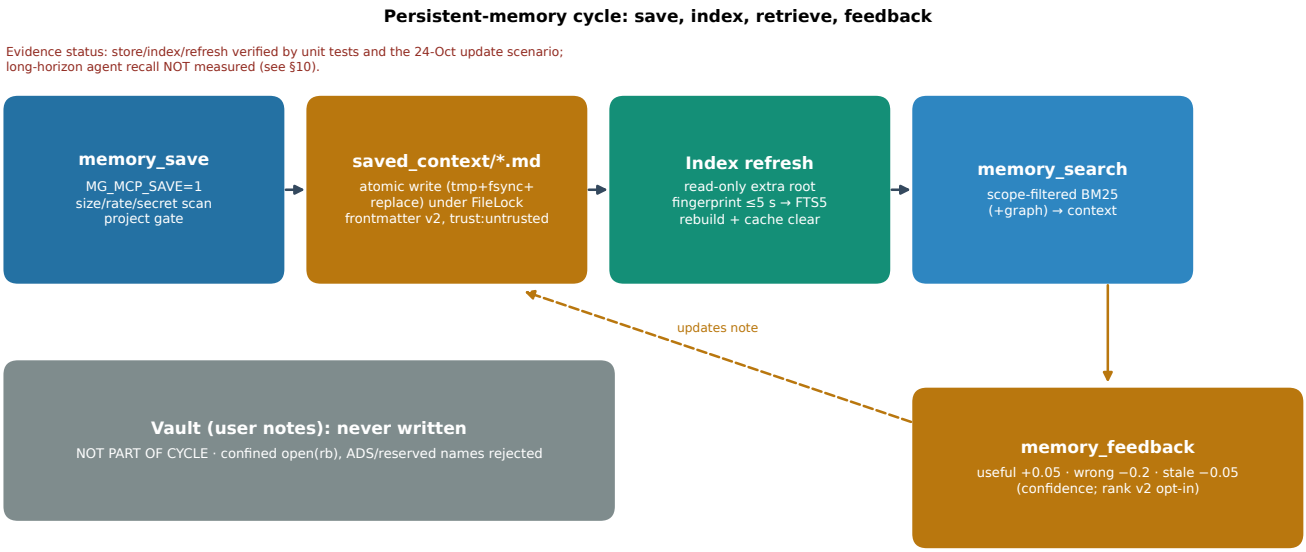


Fig. 6. Persistent-memory cycle. The mechanism is implemented; its end-to-end benefit for agents was not measured.

Isolation. Searches accept a scope such as `projeto:X`; `MG_SCOPE` can pin a session so that tools can only *narrow* it. By default scope is applied after the top-100 candidate cut, so a narrow scope can return fewer results than exist in that project. A defence-in-depth re-check counts and blocks leaked results.

Agent integration. Fig. 7 shows the communication paths. Adapters exist for Hermes, Claude Code and Codex; OpenCode is detection-only and unvalidated; we found no Cursor integration. Local and remote models are reached through a generic OpenAI-compatible client (Ollama, vLLM, OpenRouter, OpenAI, Anthropic adapters; vLLM

untested against a real server). Reads on the default loopback bind need no token; mutating routes require a loopback client, matching Origin and Referer, and `X-MG-Token`. Retrieved notes carry `trust="untrusted"`. The browser dashboard is vanilla ES modules with a hash router and no build step.

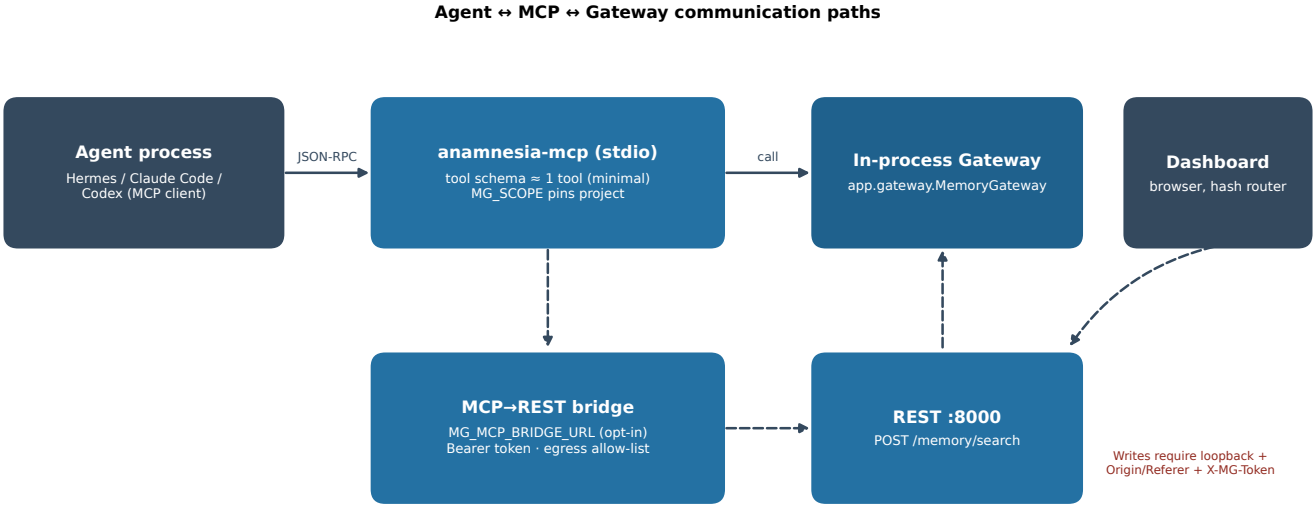


Fig. 7. Communication between agents, the MCP server and the gateway.

Experimental or unfinished. The Decision Engine presets, Colibrì (local JEV backend), LLMLingua/Provence compressors and rerankers (separate Python environment), and the SQLite note catalog are not wired into the default runtime; Laya runs on CPU but is opt-in and, per the repository's own measurements, is a weak relevance model (AUC 0.64–0.68 versus 0.975 for lexical overlap).

VI. Experimental Methodology

Scope of evidence. All results below were produced *before* this paper by two earlier executions: a retrieval-only benchmark (24 Oct 2026) and a generative benchmark (25 Oct 2026). No experiment was run for the paper; tables and figures are regenerated from the stored logs. Where we computed new statistics from stored data (paired bootstraps, stratification, a sensitivity rule), this is stated and the original verdicts remain unaltered.

Corpus and questions. *D2* is a sanitized snapshot of the author's real project notes (Portuguese, 379 notes), frozen in a copy; the original vault was never written. The 82 questions have verifiable gold sources and regular-expression answers: 36 factual, 28 numeric, 10 unanswerable, 3 ambiguous, 3 duplicate/version, 2 long-document. (A synthetic reproducible corpus, D1, exists but was not used generatively.)

Arms (Fig. 8). *Without*: an independent harness (imports of Anamnésia are blocked by a guard) with BM25 over whole notes, delivering the top-5 notes; its abstention threshold θ is calibrated on a development split, and it returned an empty context for 22 of 82 questions (10 unanswerable, 12 answerable). *With*: the real Anamnésia through MCP stdio, fresh home directory and vault copy per block, offline: **baseline** (BM25 sections), **Graphify** (hybrid), **auto+optimizer** (`pipeline=auto`, which resolves to baseline, plus the optimizer and cache), and **Graphify+JEV** (paid). Contexts were produced once (cold pass) and replayed *byte-identically* to every model, so the sole varying factor between arms is the context.

Generation. A fixed system prompt instructs the model to answer only from the context, cite sources, and otherwise reply "Não encontrei essa informação no contexto". Temperature 0, seed 42, 512 output tokens, one repetition per cell. Eight models via OpenRouter [33] (gpt-4o-mini, gpt-4.1-mini, gemini-2.5-flash-lite, deepseek-chat-v3.1, llama-3.3-70b-instruct, qwen3-235b-a22b-2507, mistral-small-3.2-24b-instruct, claude-haiku-4.5) and one local model (qwen3.5:9b via Ollama on an RTX 3060 shared with other processes). Remote cost is the provider-reported `usage.cost`; JEV cost is *calculated* (Eq. 2); no TypeSafe invoice was available.

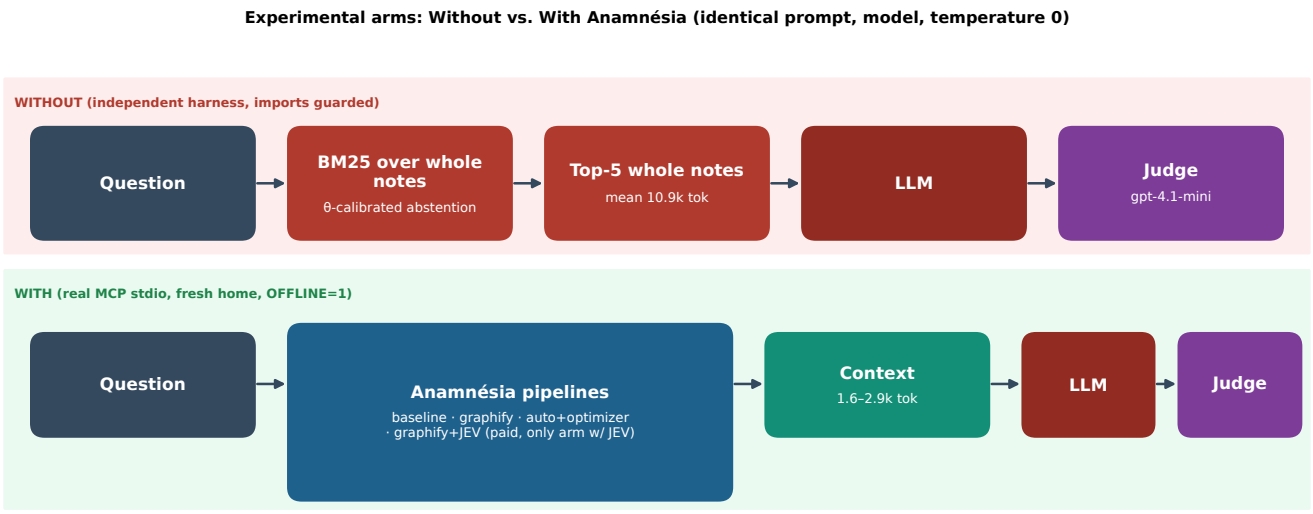


Fig. 8. Experimental design: identical prompt, model and temperature; only the context differs.

Scoring. An independent LLM judge (gpt-4.1-mini, temperature 0) compared each answer with the gold passage and returned CORRECT, INCORRECT or ABSTAIN (for unanswerable questions: ABSTAIN is correct). Accuracy is the share of CORRECT on the 72 answerable questions. Intervals for single proportions are Wilson intervals [34]; paired differences use a question-level bootstrap [35] (4,000 resamples, seed 20261025) pooling the nine models, which treats questions as the sampling unit.

Volume. 3,797 generation rows were logged, 3,690 valid (9 models $\times$ 5 arms $\times$ 82 queries) and 107 failed attempts (HTTP 429/400) that were retried until every cell had 82 answers; 3,690 judgments were produced. Generation cost was US$4.60 and judging US$0.75; the OpenRouter balance fell by about US$5.38.

Retrieval-only benchmark (24 Oct). Datasets D1 (synthetic), D2 (real) and D3 (D2 before/after a vault-structure change), 5 measured repetitions after one discarded warm-up, cold and hot passes, tiktoken `cl100k_base` for all token counts, paired bootstrap with 2,000 resamples (seed 20261024). It also includes a stronger *OFF-B* baseline: BM25 over *sections* with the same token budget as the corresponding Anamnésia run.

VII. Experimental Results

A. Tokens, cost and accuracy

Table II and Fig. 9 give the pooled results. Provider-reported input tokens fall from 11,103 (Without) to 1,776 (auto+optimizer), a reduction of 84%; the mean tiktoken size of the delivered context falls from 10,889 (maximum 71,308) to 1,637. Mean output length is about 60–70 tokens in all arms, so cost is almost entirely input cost. The effective cost of the eight remote models drops by 81.7% for auto+optimizer (per model 80–86%, Fig. 12); JEV adds US$0.0101 per 82-query run (252,404 judge tokens), yet because the JEV arm also delivers larger contexts, its saving is only 68.8% pooled (59–73% per model).

Accuracy is statistically indistinguishable from the baseline for auto+optimizer ($+0.3$ pp; Table III), and also for the plain baseline pipeline ($+2.2$ pp) and for Graphify+JEV (-1.5 pp). Graphify without JEV is the only arm with a significant loss (-15.0 pp, CI excluding zero). Four of nine models score at least as high with auto+optimizer as without. Accuracy in a single cell has a 95% CI of roughly ± 8 pp (Fig. 10); only pooled comparisons are informative.

Savings scale with model price: for claude-haiku-4.5 the cost of 82 queries falls from US$1.10 to US$0.22; for mistral-small-3.2 from US$0.072 to US$0.012, so the absolute saving is small for cheap models. For the local model there is no dollar saving (zero API cost in all arms), although its median generation latency fell from 8.1 s to 2.6 s with auto+optimizer; this comparison is confounded by GPU sharing and Ollama model loading (the first call took

TABLE II

POOLED RESULTS OVER NINE MODELS (72 ANSWERABLE QUESTIONS EACH; ACCURACY n=648 PER ARM). COST SUMS THE EIGHT REMOTE MODELS (US$ PER 82 QUERIES PER MODEL); JEV COST IS CALCULATED AND, FOR COMPARABILITY, ATTRIBUTED TO EACH MODEL (THE JUDGE CONTEXT WAS PRODUCED ONCE; ACTUAL JEV SPEND WAS ABOUT US$0.010); GENERATION COST IS MEASURED. SAVINGS ARE RELATIVE TO THE *Without* ARM.

Arm	In-tok	Acc. (%)	Cost	of which JEV	Saving	US$/correct$\times 10^3$
Without (BM25, whole notes)	11,103	86.6	2.340	0.0000	–	4.69
With: baseline	2,688	88.7	0.608	0.0000	74.0%	1.19
With: Graphify hybrid	2,460	71.6	0.571	0.0000	75.6%	1.38
With: auto + optimizer	1,776	86.9	0.428	0.0000	81.7%	0.86
With: Graphify + JEV	2,908	85.0	0.731	0.0807	68.8%	1.50

TABLE III

PAIRED DIFFERENCE IN ACCURACY VERSUS *Without* (PERCENTAGE POINTS; QUESTION-LEVEL BOOTSTRAP OVER THE 72 ANSWERABLE QUESTIONS, 4000 RESAMPLES, SEED 20261025, NINE MODELS POOLED).

Arm	Δ acc. (pp)	95% CI
With: baseline	+2.2	[-4.9, +9.4]
With: Graphify hybrid	-15.0	[-25.5, -4.2]
With: auto + optimizer	+0.3	[-7.4, +8.0]
With: Graphify + JEV	-1.5	[-10.6, +7.9]

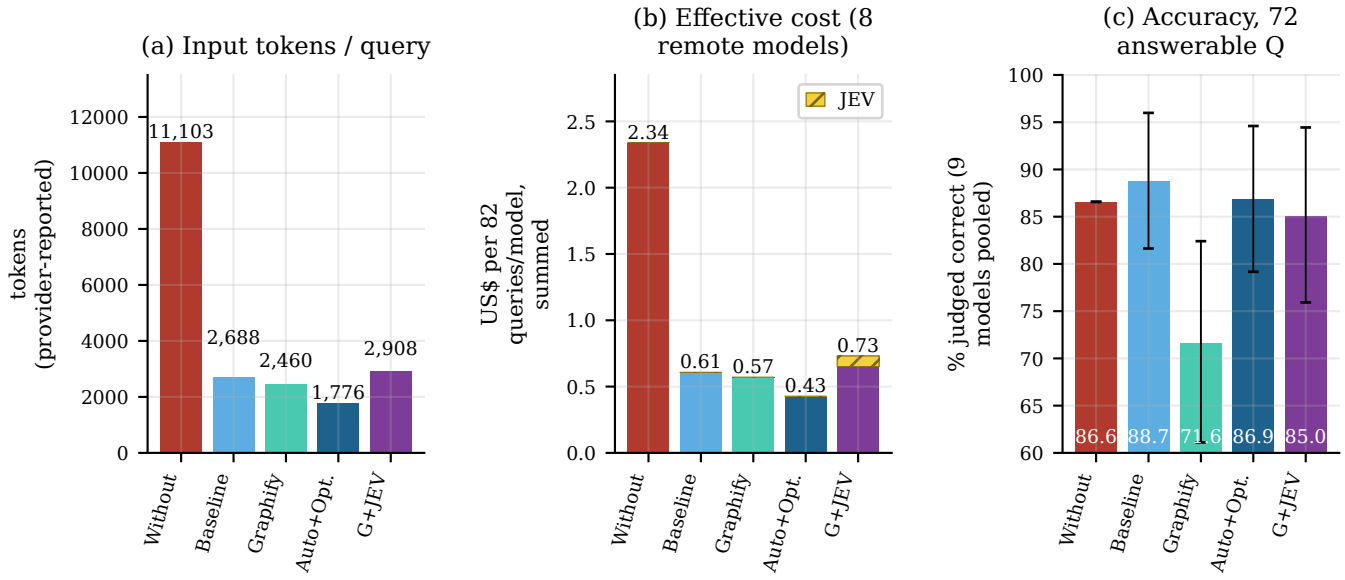


Fig. 9. Pooled headline results. (a) Provider-reported input tokens per query. (b) Effective cost over the eight remote models; hatched = JEV (calculated). (c) Accuracy on 72 answerable questions; error bars are 95% bootstrap CIs of the paired difference to *Without*, plotted around each arm's value.

TABLE IV

PER-MODEL RESULTS (SINGLE RUN, TEMPERATURE 0, 82 QUERIES). COST = MEASURED GENERATION COST (+ CALCULATED JEV FOR THE LAST COLUMN) IN US$; "SAVE" IS THE EFFECTIVE-COST SAVING VS. *Without*; ACCURACY OVER 72 ANSWERABLE QUESTIONS. THE LOCAL MODEL HAS ZERO API COST AND IS EXCLUDED FROM SAVING COLUMNS.

	Without		Baseline		Graphify		Auto+opt.		Graphify+JEV		
Model	Cost	Acc.	Save%	Acc.	Save%	Acc.	Save%	Acc.	Cost	Save%	Acc.
local:qwen3.5:9b	0	86	–	90	–	69	–	92	0.010	–	86
mistral-small-3.2-24b-instruct	0.072	88	74	88	76	74	83	85	0.030	59	85
gemini-2.5-flash-lite	0.096	88	74	92	77	71	82	83	0.036	62	83
qwen3-235b-a22b-2507	0.111	88	75	89	76	75	83	86	0.040	64	86
gpt-4o-mini	0.128	88	74	83	77	68	82	81	0.046	65	83
llama-3.3-70b-instruct	0.148	88	74	86	75	72	82	86	0.048	67	82
deepseek-chat-v3.1	0.340	83	79	93	78	74	86	88	0.090	73	89
gpt-4.1-mini	0.344	88	74	88	76	69	82	89	0.107	69	88
claude-haiku-4.5	1.102	85	72	90	74	72	80	93	0.334	70	83

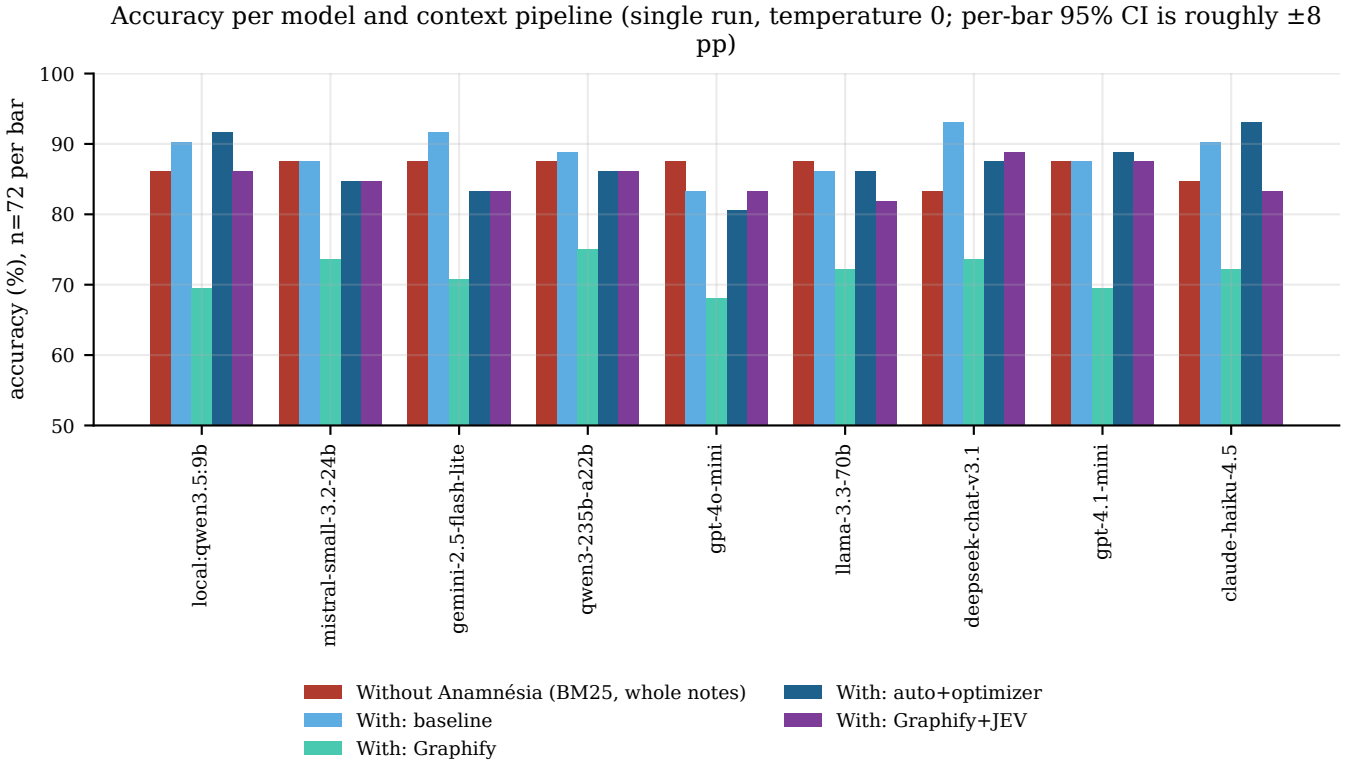


Fig. 10. Accuracy by model and pipeline (72 questions per bar, single run).

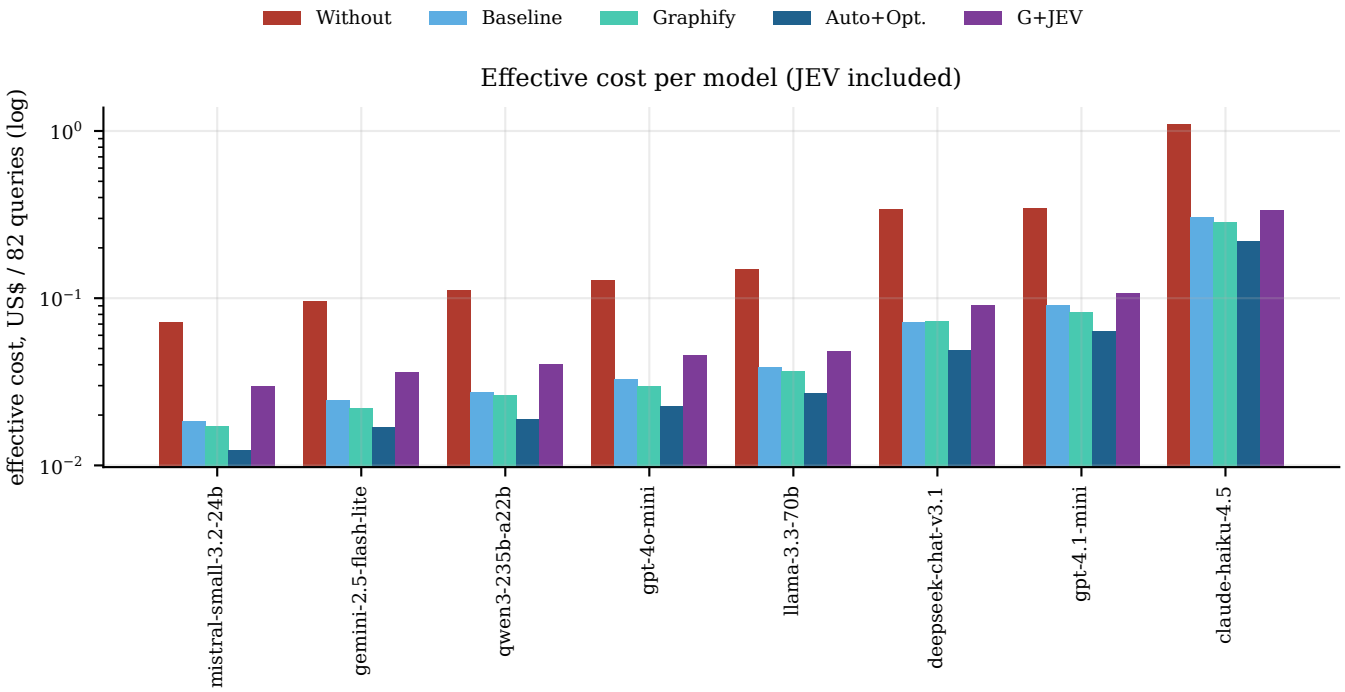


Fig. 11. Effective cost per model (log scale), JEV included.

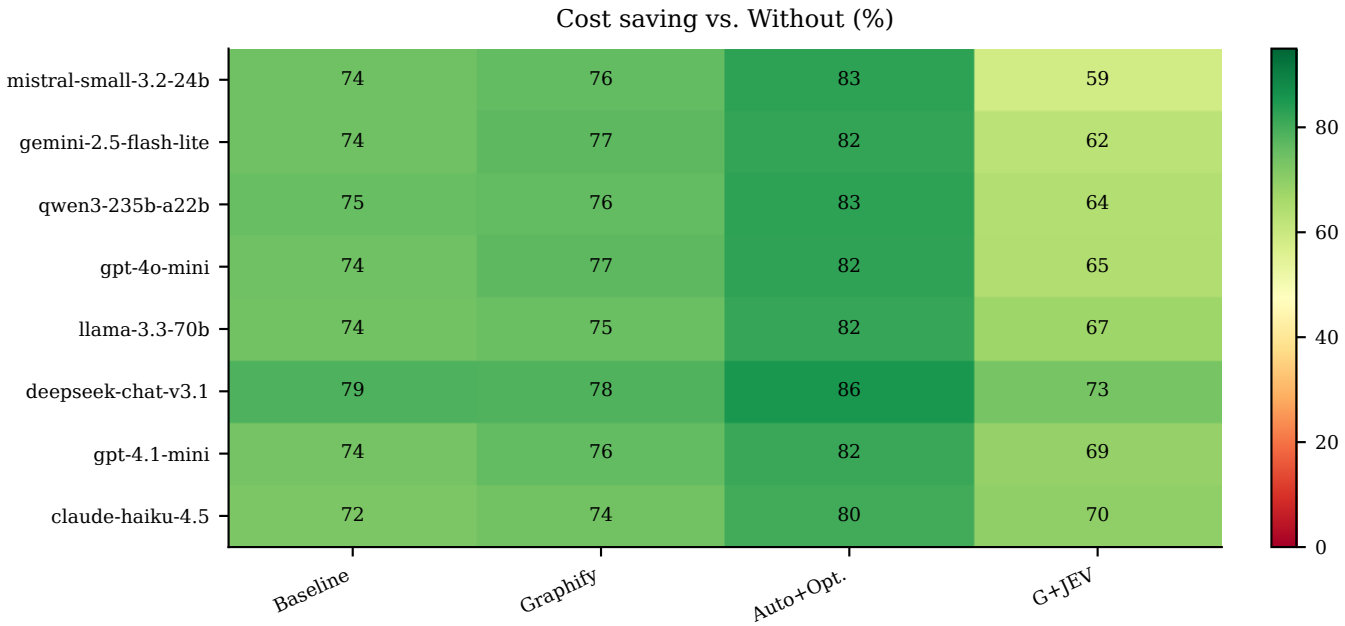


Fig. 12. Saving of effective cost relative to *Without*, by model and pipeline (%).

6.8 s). Remote generation latency changed less (mean of per-model p50: 2.98 s Without vs. 2.30 s auto+optimizer) and is dominated by provider queueing at six concurrent calls. Fig. 13 plots cost against accuracy.

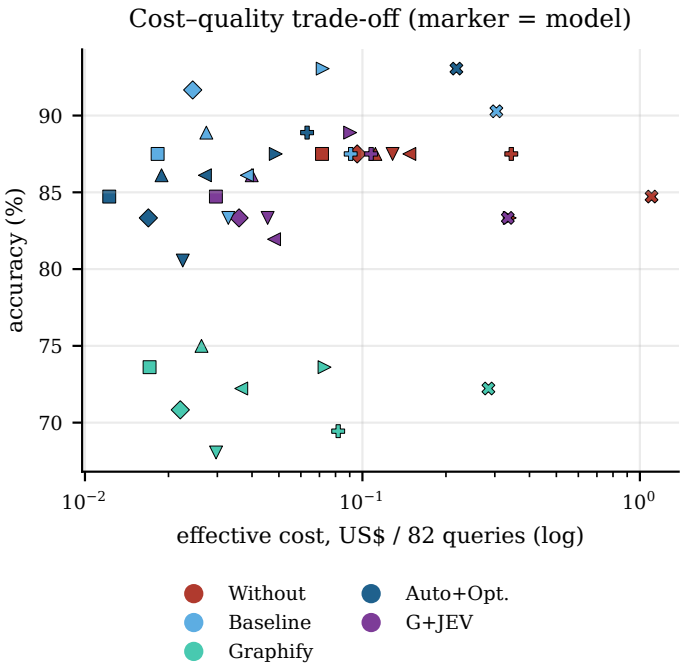


Fig. 13. Cost–quality trade-off. Marker shape identifies the model.

B. Quality by question class

Table V and Fig. 14 show that Graphify's loss is concentrated on numeric questions (57% vs. 94% Without), where the exact figure sits in note bodies that the label-only graph does not index. The gold answer was present in the delivered context for 53% of answerable questions with Graphify versus 81% (Without), 85% (baseline), 83% (auto+optimizer) and 82% (JEV), as measured by the answer regular expression on the stored contexts (a proxy). The duplicate/version class (3 questions) is the only one in which auto+optimizer is far below Without (41% vs. 67%), consistent with the retrieval benchmark's observation that some superseded `-pre-` notes are still delivered; the sample is anecdotal.

TABLE V
ACCURACY (%) BY QUESTION CLASS, NINE MODELS POOLED. CLASSES WITH $n \leq 3$ QUESTIONS ARE ANECDOTAL.

Class	n	Without	Base.	Graph.	Auto	G+JEV
factual	36	87	94	85	91	89
numeric	28	94	91	57	91	87
duplicate/version	3	67	48	67	41	67
ambiguous	3	63	70	70	67	70
long document	2	39	56	50	56	50

C. Stratified analysis and a scoring defect

The *Without* harness abstains when its top BM25 score is below θ, so for 12 of the 72 answerable questions it supplied an empty context. Splitting by this event (Table VI) reveals that the near-zero pooled difference is the sum

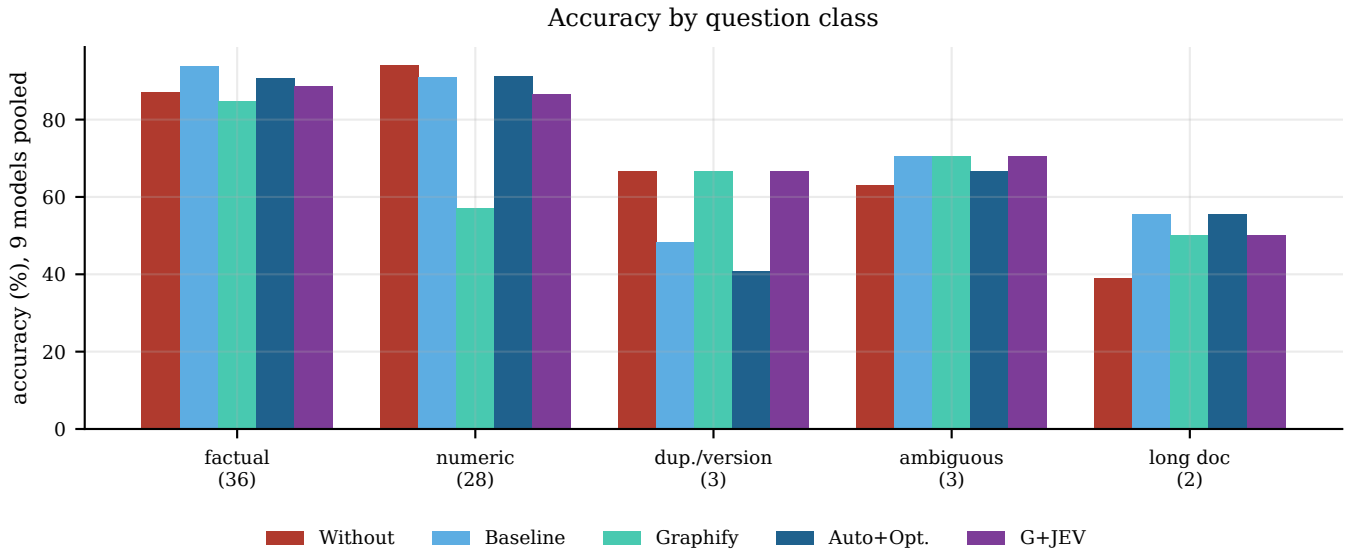


Fig. 14. Accuracy by question class (nine models pooled).

TABLE VI

STRATIFIED ACCURACY (%), NINE MODELS POOLED. *Retrieved* = THE 60 ANSWERABLE QUESTIONS FOR WHICH THE *Without* HARNESS DELIVERED A CONTEXT; *Without-empty* = THE 12 ANSWERABLE QUESTIONS ON WHICH ITS θ GATE ABSTAINED (EMPTY CONTEXT). UPPER BLOCK: VERDICTS EXACTLY AS PRODUCED BY THE LLM JUDGE. LOWER BLOCK: SENSITIVITY RULE R1 (AN ANSWER THAT BEGINS WITH THE MANDATED ABSTENTION SENTENCE IS NEVER SCORED CORRECT). Δ = PAIRED DIFFERENCE VS. *Without* IN PP WITH 95% BOOTSTRAP CI.

Scoring	Stratum (n)	Without	Baseline	Graphify	Auto+opt.	G+JEV
Judge as run	all answerable (72)	86.6	88.7	71.6	86.9	85.0
		Δ	+2.2 [−4.9,+9.4]	−15.0 [−25.5,−4.2]	+0.3 [−7.4,+8.0]	−1.5 [−10.6,+7.9]
Judge as run	retrieved (60)	94.3	89.3	68.0	87.4	83.0
		Δ	−5.0 [−10.0,−0.7]	−26.3 [−36.5,−17.0]	−6.9 [−12.4,−1.9]	−11.3 [−18.9,−4.6]
Judge as run	Without-empty (12)	48.1	86.1	89.8	84.3	95.4
		Δ	+38.0 [+10.2,+67.6]	+41.7 [+16.7,+66.7]	+36.1 [+6.5,+64.8]	+47.2 [+16.7,+75.9]
Rule R1	all answerable (72)	76.9	84.0	55.9	81.6	80.2
		Δ	+7.1 [−2.6,+16.5]	−21.0 [−35.8,−6.0]	+4.8 [−5.6,+15.3]	+3.4 [−9.4,+15.7]
Rule R1	retrieved (60)	92.2	85.6	50.6	83.3	77.8
		Δ	−6.7 [−14.3,−0.4]	−41.7 [−53.7,−29.3]	−8.9 [−17.0,−1.7]	−14.4 [−23.7,−5.6]
Rule R1	Without-empty (12)	0.0	75.9	82.4	73.1	92.6
		Δ	+75.9 [+54.6,+94.4]	+82.4 [+62.0,+98.1]	+73.1 [+49.1,+92.6]	+92.6 [+77.8,+100.0]

of two opposite effects. On the 60 questions where *Without* retrieved something, it scored 94.3% against 87.4% for auto+optimizer (paired -6.9 pp, CI $[-12.4, -1.9]$), 89.3% for baseline (-5.0, CI $[-10.0, -0.7]$) and 83.0% for JEV (-11.3). On the 12 questions with empty *Without* context, Anamnésia was far better (+36 to +47 pp).

The *Without* score on those 12 questions (48.1%) is implausibly high for an answer that should be an abstention. Inspection of stored answers showed that the judge labelled some replies consisting only of the mandated abstention sentence as CORRECT, despite instructions. Since this is a defect of our evaluation, we report a deterministic sensitivity rule R1 (an answer beginning with the abstention sentence is never CORRECT on an answerable question) applied to the stored text; it changes 63 *Without* verdicts and between 31 and 102 per Anamnésia arm. Under R1 pooled accuracy becomes 76.9% (Without), 84.0% (baseline), 55.9% (Graphify), 81.6% (auto+optimizer) and 80.2% (JEV); the pooled paired differences ($+4.8$ for auto+optimizer, CI $[-5.6, +15.3]$) remain statistically indistinguishable from zero, but the stratified pattern persists (-8.9 pp on the 60 retrieved questions, CI $[-17.0, -1.7]$). We therefore regard the following as the best-supported reading: *Anamnésia reduces tokens by about 84% and is more robust than a thresholded whole-note BM25 when that BM25 retrieves nothing, but when the whole-note BM25 does retrieve, giving the model entire notes yields measurably higher accuracy than the trimmed context*. The headline "86.9% vs. 86.6%" is accurate as the as-run judged value, but it should not be read as "no quality cost".

D. Unanswerable questions

All nine models in all five arms abstained correctly on all ten unanswerable questions, as judged. This reflects the instruction-following of the generator, not the gateway: the retrieval benchmark shows that Anamnésia *always* returns a context (abstention rate 0% on the unanswerable set, false-positive rate 100%), whereas the thresholded *Without* harness abstains correctly 100% of the time but also wrongly abstains on 16.7% of answerable questions.

E. Retrieval-only evidence (24 Oct, real measurements)

Table VII and Fig. 15 summarize D2. Against *Without* (OFF-A), the Anamnésia pipelines deliver 77–86% fewer context tokens at higher recall@10 (0.97 vs. 0.83); Graphify's gold-answer-in-context rate (0.56) is far below baseline (0.83), matching the generative loss. Against the stronger OFF-B (section-level BM25 at the same budget), Anamnésia delivers 8–10% *more* tokens (CI includes zero) but higher recall@10 (paired $\Delta = +0.21$ to $+0.24$, CIs excluding zero) and higher answer-in-context ($+0.10$ baseline; $+0.135$ auto, CI $[0.02, 0.25]$). Thus against a competent chunked baseline the advantage is retrieval quality, not token cost; the generative benchmark did not include OFF-B. Latency (Fig. 16): auto+optimizer has p50 of 45.6 ms cold and 9.9 ms hot (result cache), versus 0.7 ms for the in-process baseline, i.e. tens of times slower but negligible relative to LLM latency of seconds; Laya (local CPU) needs ≈ 4 s per query and delivered 6.7k tokens (40% reduction) with recall@10 of 0.75.

TABLE VII
RETRIEVAL-ONLY BENCHMARK OF 24 OCT 2026 (D2, TEST SPLIT, 56 QUERIES/52 ANSWERABLE, 5 MEASURED REPETITIONS, TIKTOKEN `cl100k_base`). LATENCY IS P50 OF THE CLIENT ROUND TRIP (MCP STDIO FOR *With*; IN-PROCESS FOR *Without*).

Arm	Ctx tok	R@5	R@10	Ans. in ctx	Cold ms	Hot ms
Without (OFF-A)	11,207	0.77	0.83	0.83	0.7	0.7
Section BM25 @ auto budget (OFF-B)	1,437	0.67	0.73	0.67	5.6	4.3
Baseline	2,541	0.89	0.97	0.83	21.9	21.6
Graphify hybrid	2,333	0.89	0.97	0.56	14.7	12.3
Auto + optimizer	1,589	0.85	0.97	0.81	45.6	9.9
Laya (local, CPU)	6,715	0.73	0.75	0.83	3970.9	4062.6

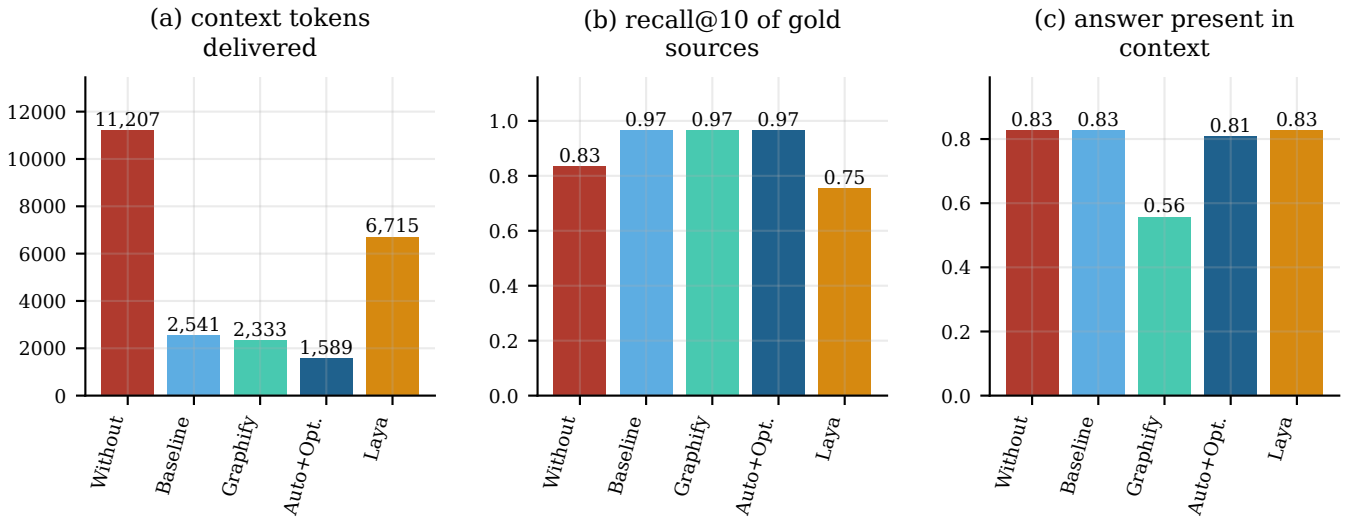


Fig. 15. Retrieval-only benchmark on D2 (test split): context tokens, recall@10 and presence of the answer in the delivered context.

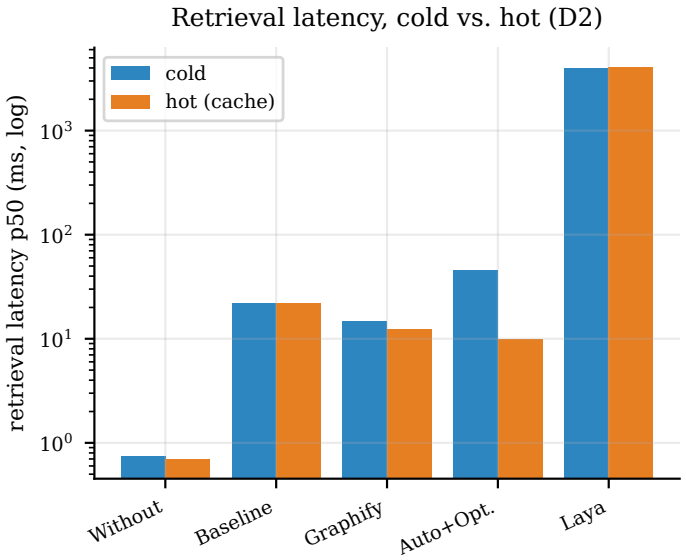


Fig. 16. Retrieval latency, cold vs. hot.

F. Persistence, update and isolation scenarios (24 Oct)

Small controlled scenarios exercised the memory mechanisms. (i) *Save and scoped search* (F9-01): 18/18 checks passed for the three Anamnésia pipelines (a note saved under a project is found in its scope and not in another). (ii) *Isolation* on the synthetic corpus: without scope, 68–82% of delivered sources came from other projects; with `projeto:X`, 0% leakage at hit@10 ≥ 0.97. (iii) *Update with the server running*: 3 s after editing a note the old version was still served; at 7 s the new version was served by baseline and auto, but the Graphify arm did not reflect the edit at either time (the graph is built per session). (iv) *Injection/disinformation* (N=5 and 3): the decoy note was delivered by every pipeline; the optimizer flagged injection in 5/5 and misinformation in 2/3 cases without blocking. (v) D3 (before/after a structural change of the vault, 67 paired questions): recall@10 and answer-in-context unchanged ($\Delta = 0$), recall@5 changes of -0.5 to -3.5 pp with CIs touching zero. These are *mechanism checks with small N*; they do not establish that an agent remembers earlier conversations.

VIII. Discussion

What is demonstrated. (1) The gateway reliably shrinks the prompt by roughly 84% against whole-note BM25 and, with the model's own pricing, converts this into a cost saving of 80–86% for the default pipeline across eight remote models (measured generation cost; zero retrieval dollars). (2) Pooled accuracy as judged is not distinguishable from the whole-note baseline, for the default pipeline and for plain baseline retrieval. (3) Retrieval quality of the default pipeline exceeds both whole-note and section-level BM25 on recall of gold sources.

What is not shown, or is contradicted. (a) The paid judge did not pay for itself here: it added US$0.08 across eight runs and enlarged contexts (2,908 vs. 1,776 tokens), without accuracy gain (85.0% vs. 86.9%). One plausible reason is that the JEV pipeline starts from the Graphify candidate set, whose recall of answer-bearing sections is low; we have not isolated this. (b) Graphify's graph is the wrong tool for *finding* text: it holds labels, not bodies, and the rank-ladder scores interact poorly with the 0.5 adaptive cut. The hybrid fixes recall of *files* (recall@10 0.97) but not of *answer-bearing passages* (0.56). (c) The headline parity hides a stratified deficit versus whole-note BM25 when the latter retrieves (Section VII-C).

Why trimmed context can lose. An adaptive cut and a 300-token snippet cap may remove the exact sentence that contains a number; whole notes include it. The loss on numeric questions for Graphify (57%) supports this view; for baseline and auto it is a few points. Tuning the snippet and budget parameters on a development split, and adding section-level whole-note fallbacks, are natural remedies that we have not tried.

Where cost savings matter. Absolute savings are material only for priced models and long contexts; for cheap models the JEV overhead consumes up to a third of the savings (mistral: 83% $\rightarrow$ 59%). For local models the benefit is latency and context-window pressure, not dollars.

IX. Limitations and Threats to Validity

Internal validity. One repetition per cell; per-cell 95% CIs are about ± 8 pp. The judge is a single LLM, was not validated against humans, and exhibited the abstention-scoring defect of Section VII-C; R1 is a heuristic correction, not a re-judgement. The *Without* arm's empty-context gate (θ) is itself a design choice that interacts with accuracy. The gold regular expressions were written against contexts rather than free-form answers and are used only for context-presence analysis. Judge, generator and some evaluated models belong to the same vendor families, so self-preference cannot be excluded.

Baseline strength. *Without* delivers whole notes (mean 10.9k tokens, maximum 71k), which is a weak baseline for cost; a section-level BM25 (OFF-B) was evaluated for retrieval only, not generation. Part of the token saving is attributable to chunking, which is not unique to Anamnésia.

Construct validity. Cost is token cost; energy, engineering time and the cost of maintaining the gateway are excluded. JEV cost is calculated from a published tariff and token counts, not invoiced. Provider tokenizers differ, so "input tokens" are not comparable across models in absolute terms.

External validity. One Portuguese-language, single-author engineering vault (379 notes); 82 questions; a 2026 model snapshot. Results may not transfer to other languages, corpora, or to a vault with heavy paraphrase where lexical retrieval is weak.

Not evaluated. Persistence across real agent sessions, recall after long multi-turn conversations, memory consolidation, end-to-end task success, comparison with dense or graph-RAG systems, the Decision Engine, Laya in generation, hybrid/ensemble strategies without a retrieval route, premium models, and providers whose keys exist but which were not tested. Deployed claims in the project README (for instance "$-39.7\%/-24.2\%$ context at unchanged recall" and schema-token comparisons) have no stored artifact in the audited material, and the 24 Oct benchmark shows recall@5 on D1 dropping from 0.903 to 0.799 for the optimizer arm while recall@10 is unchanged.

Security. The injection screen is heuristic and non-blocking; retrieved notes are labelled untrusted but the consuming agent decides what to do with them. Write endpoints are loopback- and token-guarded; reads on loopback require no token.

X. Future Work

1) **Long-horizon memory evaluation**: run LoCoMo- and LongMemEval-style tasks [19], [20] with real agents across sessions, measuring recall of decisions after dozens of turns with and without `memory_save`.
2) **Fair generative baselines**: add OFF-B (section BM25 at equal budget) and a dense retriever to the generative study; add repetitions and human-validated judging.
3) **Parameter tuning**: snippet cap, adaptive-cut ratio and budget tuned on a development split; section-level fallbacks for numeric questions.
4) **A self-contained graph**: generate the link/heading graph inside Anamnésia (wikilinks, headings, frontmatter) and measure recall against the Graphify graph, removing the external dependency for graph construction.
5) **Judge economics**: evaluate JEV on a candidate set with higher recall (baseline rather than Graphify seeds), with invoiced rather than calculated cost.
6) **Freshness for graphs**: incremental graph refresh so edits become visible within the vault refresh interval.
7) **Stronger injection defences**: blocking policies and learned detectors, evaluated on a larger adversarial set.

XI. Conclusion

Anamnésia is a working, local, agent-independent memory gateway that implements lexical retrieval, graph-enhanced retrieval, context trimming, near-duplicate removal, caching, injection flagging and cost ledgering behind MCP and REST interfaces. In a real, fully logged benchmark, its default pipeline reduced input tokens by about 84% and effective cost by about 82% across eight remote models with pooled accuracy statistically indistinguishable from a whole-note BM25 baseline; but a stratified re-analysis shows lower accuracy when that baseline did retrieve, the Graphify pipeline hurt accuracy by 15 pp, and the paid JEV judge was not cost-effective in this setting. Whether the persistent-memory design improves agent continuity over real, long sessions remains an open empirical question that this paper deliberately does not answer.

Availability

The public package `assets/paper-v2.1.0/` contains the LaTeX sources, English and Portuguese PDFs, bilingual figures, aggregate figure data and plotting scripts. The original generation and evaluation logs (`calls.jsonl`, `judge.jsonl`), the private vault snapshot and the original scoring inputs remain local under `data/reports-out/`; they are not included in the public package. The published aggregates reproduce the figures, not the full generative experiment. This paper adds no new model experiments or API expenditure.

Acknowledgments

We thank the maintainers of Graphify, SQLite, FastAPI, the Model Context Protocol, Ollama, OpenRouter and TypeSafe AI, whose work Anamnésia builds on.

REFERENCES

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, arXiv:2005.11401. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997

[4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[5] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th ACM UIST*, 2023.

[6] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI*, 2024, arXiv:2305.10250. [Online]. Available: https://arxiv.org/abs/2305.10250

[7] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," *arXiv preprint arXiv:2504.19413*, 2025. [Online]. Available: https://arxiv.org/abs/2504.19413

[8] SQLite Consortium, "SQLite FTS5 extension (including the `bm25()` auxiliary function)," https://www.sqlite.org/fts5.html, 2026, accessed 25 Oct 2026.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[10] Graphify Labs, "Graphify: turn a folder of code and documents into a queryable knowledge graph (pypi package `graphifyy` 0.9.59, apache-2.0)," https://github.com/Graphify-Labs/graphify, 2026, accessed 25 Oct 2026.

[11] TypeSafe AI, "TypeSafe api and models documentation (jev, `jev-1.13.0`; python sdk `typesafe-sdk` 0.7.1)," https://docs.typesafe.ai, 2026, accessed 25 Oct 2026.

[12] Model Context Protocol contributors, "Model context protocol specification (2025-06-18)," https://modelcontextprotocol.io/specification/2025-06-18, 2025, accessed 25 Oct 2026.

[13] S. Ramírez, "FastAPI framework documentation," https://fastapi.tiangolo.com, 2026, accessed 25 Oct 2026.

[14] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, arXiv:2004.04906. [Online]. Available: https://arxiv.org/abs/2004.04906

[15] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd ACM SIGIR*, 2009.

[16] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[17] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2405.14831. [Online]. Available: https://arxiv.org/abs/2405.14831

[18] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999-66, 1999. [Online]. Available: http://ilpubs.stanford.edu:8090/422/

[19] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. ACL*, 2024, arXiv:2402.17753. [Online]. Available: https://arxiv.org/abs/2402.17753

[20] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. ICLR*, 2025, arXiv:2410.10813. [Online]. Available: https://arxiv.org/abs/2410.10813

[21] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. EMNLP*, 2023.

[22] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," *arXiv preprint arXiv:2310.06839*, 2023. [Online]. Available: https://arxiv.org/abs/2310.06839

[23] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[24] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024. [Online]. Available: https://arxiv.org/abs/2406.18665

[25] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th ACM STOC*, 2002.

[26] G. S. Manku, A. Jain, and A. Das Sarma, "Detecting near-duplicates for web crawling," in *Proc. 16th WWW*, 2007.

[27] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997.

[28] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 2023, arXiv:2306.05685. [Online]. Available: https://arxiv.org/abs/2306.05685

[29] Ollama contributors, "Ollama: run large language models locally," https://github.com/ollama/ollama, 2026, accessed 25 Oct 2026.

[30] Convai Innovations, "Laya: a fast, non-autoregressive system 1 decision engine (pypi `laya`, apache-2.0)," https://huggingface.co/convaiinnovations/laya, 2026, accessed 25 Oct 2026.

[31] Pydantic Services Inc., "Pydantic documentation," https://docs.pydantic.dev, 2026, accessed 25 Oct 2026.

[32] OpenAI, "tiktoken: a fast BPE tokeniser," https://github.com/openai/tiktoken, 2026, accessed 25 Oct 2026.

[33] OpenRouter, Inc., "OpenRouter api documentation," https://openrouter.ai/docs, 2026, accessed 25 Oct 2026.
[34] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
[35] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.